UNDERGRADUATE COMPUTING
M255 Object-oriented programming with Java

M255

Software Guide – Java Reference

SUP 98776 9

2.1

Contents

Introduction **3**

1 OU Class Library **4**

2 M255 example classes **7**

2.1 Amphibian classes 7

2.2 Barndancing 10

2.3 Account classes 11

2.4 Shapes classes 14

3 Strings **20**

4 Java Collections Framework **25**

4.1 Collection interfaces 25

4.2 Collection implementation classes 31

4.3 Collection utility classes 33

5 Files and streams **37**

5.1 Files and pathnames 37

5.2 Reading from character streams 38

5.3 Writing to character streams 41

5.4 Reading from byte streams 43

5.5 Writing to byte streams 45

5.6 Serialisation 47

6 Exceptions **48**

6.1 Checked exceptions 48

6.2 Unchecked exceptions 49

7 Java operators used in M255 **52**

Index **53**

Introduction

This booklet provides a reference to the classes and interfaces you have encountered in M255. It is based on the Javadoc for those classes and interfaces, but differs from the Javadoc in three respects.

- For the M255 example classes, private instance variables *are* shown.
- In Sections 3, 4 and 5 we have simplified the class comments and method comments and, in places, the method headings. This documentation is sufficient for your study of M255.
- For each class or interface we do not necessarily show every method that is available, we omit some methods that are not needed for your study of M255. If you want more detail you can access the full documentation for the Java Class Libraries from BlueJ's Help menu.

This booklet is not designed to be read from cover to cover, rather you should use its index to find the documentation for a particular method, class or interface.

Use of the Java Reference in the examination

You will be allowed to take this booklet into the examination, however the following conditions apply.

- You can *only* take the version of this booklet that was printed and sent to you by the Open University.
- This booklet must be free of any annotations, except for any errata that may appear on the M255 website.

The PDF for this booklet is available on the M255 website, so you are free to print your own copy of this booklet and annotate it how you wish, but remember such a copy *cannot* be taken into the examination.

1 OU Class Library

Class OUAnimatedObject

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
public abstract class OUAnimatedObject extends Observable implements Cloneable
```

`OUAnimatedObject` is the superclass (either direct or indirect) of all classes used in M255 whose objects have a graphical representation in the Graphical Display.

Method Summary

`void performAction(String action)`

Notifies all Observers of the `OUAnimatedObject` that it has performed the action defined by `action`.

`void update(String instanceVariable)`

Notifies all Observers of the `OUAnimatedObject` that the instance variable corresponding to `instanceVariable` has been changed.

Class OUColour

```
java.lang.Object
  ∟ java.awt.Color
    ∟ ou.OUColour
public class OUColour extends java.awt.Color
```

Extends `java.awt.Color` to include an improved `toString()` method and the static colours `BROWN` and `PURPLE`.

Instance variables

`static OUColour BLACK`

The colour black.

`static OUColour BLUE`

The colour blue.

`static OUColour BROWN`

The colour brown.

`static OUColour CYAN`

The colour cyan.

`static OUColour GREEN`

The colour green.

`static OUColour MAGENTA`

The colour magenta.

`static OUColour ORANGE`

The colour orange

`static OUColour PINK`

The colour pink.

`static OUColour PURPLE`

The colour purple

`static OUColour RED`

The colour red.

`static OUColour WHITE`

The colour white.

`static OUColour YELLOW`

The colour yellow.

Constructor Summary

`OUColour(int r, int g, int b)`

Initialises the `OUColour` object with the specified red, green, and blue values in the range (0 - 255).

Method Summary

`String toString()`

Returns the string representing the colour.

Class OUDialog

```
java.lang.Object
  ∟ java.awt.Component
    ∟ java.awt.Container
      ∟ java.awt.Window
        ∟ java.awt.Dialog
          ∟ javax.swing.JDialog
            ∟ ou.OUDialog
```

`public class OUDialog extends javax.swing.Dialog`

`OUDialog` provides static methods to display dialogue boxes either to display results, or request information.

Method Summary

`static void alert(String prompt)`

Displays a dialogue box with the message `prompt` and an OK button.

`static boolean confirm(String prompt)`

Displays a yes/no dialogue box with the question `prompt`.

`static String request(String prompt)`

Displays a dialogue box with the question `prompt`, OK and Cancel buttons, and an edit box for data entry.

`static String request(String prompt, String initialAnswer)`

Displays a dialogue box with the question `prompt`, OK and Cancel buttons, and an edit box for data entry that contains the default answer `initialAnswer`.

Class OUFileChooser

```
java.lang.Object
  ∟ java.awt.Component
    ∟ java.awt.Container
      ∟ javax.swing.JComponent
        ∟ javax.swing.JFileChooser
          ∟ ou.OUFileChooser
public class OUFileChooser extends javax.swing.JFileChooser
```

`OUFileChooser` provides static methods to allow the user to select a file for input or output.

Method Summary

`static String getFilename()`

Displays a file chooser dialogue with OK and Cancel buttons allowing the user to browse the file hierarchy and select a file. Returns a string representing the selected file's pathname.

`static String getFilename(String fileName)`

Returns a string representing the pathname of a file in the current folder with the name `fileName`.

---

2 M255 example classes

2.1 Amphibian classes

The amphibian classes documented in this section are the refactored versions discussed in Unit 6, Section 4.1 and which first appear in Unit6_Project_14.

Various subclasses of amphibians such as `BovverFrog` and `ChargeableFrog` are introduced in the course as a means of investigating and demonstrating various object-oriented concepts. We do not provide documentation for such classes here as they are only used briefly in a particular unit and so do not pervade the course.

Class Amphibian

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
      ∟ Amphibian
```

Direct Known Subclasses: `Frog`, `Toad`

```
public abstract class Amphibian extends ou.OUAnimatedObject
```

The abstract class `Amphibian` is the superclass (either direct or indirect) of all M255 amphibian-like classes.

Instance variables

```
private OUColour colour
private int position
```

Method Summary

`void brown()`

Sets the colour of the receiver to brown.

`void croak()`

Causes user interface to emit a sound.

`OUColour getColour()`

Returns the colour of the receiver.

`int getPosition()`

Returns the position of the receiver.

`void green()`

Sets the colour of the receiver to green.

`abstract void home()`

Resets the receiver to its ‘home’ position.

`abstract void left()`

Moves the receiver to the left.

`abstract void right()`

Moves the receiver to the right.

`void sameColourAs(Amphibian anAmphibian)`

Sets the colour of the receiver to the argument’s colour.

`void setColour(OUColour aColour)`

Sets the colour of the receiver to the value of the argument `aColour`.

`void setPosition(int aPosition)`

Sets the position of the receiver to the value of the argument `aPosition`.

`String toString()`

Returns a string representation of the receiver.

See also the superclass `OUAnimatedObject`.

Class Frog

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
      ∟ Amphibian
        ∟ Frog
```

Direct Known Subclass: `HoverFrog`

`public class Frog extends Amphibian`

The class `Frog` defines an amphibian with the characteristics of a frog.

Constructor Summary

`Frog()`

Constructor for objects of class `Frog` that initialises `colour` to `OUColour.GREEN` and `position` to `1`.

Method Summary

`void home()`

Resets the receiver to its 'home' `position` of `1`.

`void jump()`

Causes a change in an appropriate observing user interface.

`void left()`

Decrements the `position` of the receiver by `1`.

`void right()`

Increments the `position` of the receiver by `1`.

See also the superclasses `Amphibian` and `OUAnimatedObject`.

Class HoverFrog

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
      ∟ Amphibian
        ∟ Frog
          ∟ HoverFrog
public class HoverFrog extends Frog
```

The class `HoverFrog` is a subclass of `Frog` with the additional instance variable `height` and an additional protocol to initialise and use `height`.

Instance variables

```
private int height
```

Constructor Summary

`HoverFrog()`

Constructor for objects of class `HoverFrog` that initialises `height` to `0`.

Method Summary

`void down()`

If the `height` of the receiver is greater than `0`, decrements the `height` of the receiver by `1`, otherwise the method does nothing.

`void downBy(int stepChange)`

Decreases the `height` of the receiver by the value of the argument `stepChange`. The new `height` of the receiver must lie in the range `0-6` otherwise the method does nothing.

`int getHeight()`

Returns the `height` of the receiver.

`void home()`

Resets the receiver to its 'home' `position` of `1` and to a `height` of `0`.

`void setHeight(int aHeight)`

Sets the `height` of the receiver to the value of the argument `aHeight`. `aHeight` must lie in the range `0-6` otherwise the method does nothing.

`String toString()`

Returns a string representation of the receiver.

`void up()`

If the `height` of the receiver is less than `6`, increments the `height` of the receiver by `1`, otherwise the method does nothing.

`void upBy(int stepChange)`

Increases the `height` of the receiver by the value of the argument `stepChange`. The new height of the receiver must lie in the range `0-6` otherwise the method does nothing.

See also the superclasses `Frog`, `Amphibian` and `OUAnimatedObject`.

Class Toad

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
      ∟ Amphibian
        ∟ Toad
public class Toad extends Amphibian
```

The class `Toad` defines an amphibian with the characteristics of a toad.

Constructor Summary

`Toad()`

Constructor for objects of class `Toad` that initialises `colour` to `OUColour.BROWN` and `position` to `11`.

Method Summary

`void home()`

Resets the receiver to its 'home' `position` of `11`.

`void left()`

Decrements the `position` of the receiver by `2`.

`void right()`

Increments the `position` of the receiver by `2`.

See also the superclasses `Amphibian` and `OUAnimatedObject`.

2.2 Barndancing

The `BarnDanceCaller` class was introduced in Unit 7, Section 2.3.

Class BarnDanceCaller

```
java.lang.Object
  ∟ BarnDanceCaller
public class BarnDanceCaller extends Object
```

Instances of the `BarnDanceCaller` class orchestrate dances between two `Frog` objects, `dancer1` and `dancer2`.

Instance variables

```
private Frog dancer1
private Frog dancer2
```

Constructor Summary

`BarnDanceCaller()`

Instances of the `BarndanceCaller` class orchestrate dances between two `Frog` objects, `dancer1` and `dancer2`.

Method Summary

`void dance1(int aNumber)`

Causes the dancers to first take up their positions (see `takeUpYourPositions()`), and then to perform a dance sequence of right, right, jump, left which is repeated `aNumber` of times before they turn back to green. The dancers take turns to make each move.

`void dance2(int aPosition)`

Causes the dancers to move as in `dance1()`, except they stop when a given stone, represented by the argument `aPosition` is reached. If the argument supplied represents a stone that is to the left of the central stone the dancers merely move to the central stone, turn red, then turn back to green.

`void dance3(int aNumber)`

Cause the frog objects identified by `dancer1` and `dancer2` to perform `dance1()` `aNumber` of times and then return directly to the stones they occupied before the dance began.

`Frog getDancer1()`

Returns the receiver's `dancer1`.

`Frog getDancer2()`

Returns the receiver's `dancer2`.

`void setDancer1(Frog aFrog)`

Sets the receiver's `dancer1` to the argument `aFrog`.

`void setDancer2(Frog aFrog)`

Sets the receiver's `dancer2` to the argument `aFrog`.

`void setDancers(Frog frog1, Frog frog2)`

Sets the receiver's dancers to the arguments `frog1` and `frog2`.

`void takeUpYourPositions()`

Causes both dancers to take up their positions prior to a dance (that is hop to the central stone and turn red).

2.3 Account classes

The `Account` class documented here is the version discussed in Unit 6, Section 2.2 and first seen in a complete form in Unit6_Project_4. The `CurrentAccount` class documented here is the version developed in Unit 6, Section 3 and first seen in a complete form in Unit6_Project_12_Sol.

Class Account

```
java.lang.Object
  ∟ Account
```

Direct Known Subclass: `CurrentAccount`

```
public class Account extends Object
```

The `Account` class models simple bank accounts, allowing money to be credited to, and debited and transferred from, an account.

Instance variables

```
private String holder
private String number
private double balance
```

Constructor Summary

`Account()`

Constructor for objects of class `Account`. Sets `holder` and `number` to empty strings and `balance` to `0.0`.

`Account(String holderName, String accountNumber, double anAmount)`

Constructor for objects of class `Account`, which sets the values of the `holder`, `number` and `balance` of the receiver to the arguments `holderName`, `accountNumber` and `anAmount` respectively.

Method Summary

`void credit(double anAmount)`

Credits the receiver with the value of the argument `anAmount`.

`boolean debit(double anAmount)`

If the `balance` of the receiver is equal to or greater than the argument `anAmount`, the `balance` of the receiver is debited by the argument `anAmount` and the method returns `true`, otherwise `false` is returned.

`boolean equals(Account anAccount)`

Returns `true` if the receiver is equivalent to (has the same state as) the argument `anAccount`, otherwise `false` is returned.

`double getBalance()`

Returns the `balance` of the receiver.

`String getHolder()`

Returns the `holder` of the receiver.

`String getNumber()`

Returns the `number` of the receiver.

`void setBalance(double anAmount)`

Sets the `balance` of the receiver to the value of the argument `anAmount`.

`void setHolder(String accountHolder)`

Sets the `holder` of the receiver to the value of the argument `accountHolder`.

`void setNumber(String accountNumber)`

Sets the `number` of the receiver to the value of the argument `accountNumber`.

`boolean transfer(Account toAccount, double anAmount)`

If the `balance` of the receiver is equal to or greater than the argument `anAmount`, the `balance` of the receiver is debited by the argument `anAmount`. The argument `anAccount` is then credited by the argument `anAmount` and the method returns `true`, otherwise `false` is returned.

Class CurrentAccount

```
java.lang.Object
  ∟ Account
    ∟ CurrentAccount
public class CurrentAccount extends Account
```

The `CurrentAccount` class models simple current accounts, allowing money to be credited to, and debited and transferred from, an account, subject to a given credit limit.

Instance variables

```
private double creditLimit;
private String pinNo;
```

Constructor Summary

`CurrentAccount()`

Constructor for objects of class `CurrentAccount`. Sets `creditLimit` to `0.0` and `pinNo` to `"0000"`.

```
CurrentAccount(String holderName, String accountNumber, double anAmount,
               double aLimit, String aPin)
```

Constructor for objects of class `CurrentAccount`, which sets the values of the `holder`, `number`, `balance`, `creditLimit` and `pinNo` of the receiver to the arguments `holderName`, `accountNumber`, `anAmount`, `aLimit` and `aPin` respectively.

Method Summary

`double availableToSpend()`

Calculates and returns the amount available to spend (the total of `balance` and `creditLimit`).

`boolean checkPin(String aPin)`

Returns `true` if the `pinNo` of the receiver matches the argument `aPin`, otherwise `false` is returned.

`boolean debit(double anAmount)`

If the amount available to spend (the total of `balance` and `creditLimit`) is equal to or greater than the argument `anAmount`, the `balance` of the receiver is debited by the argument `anAmount` and the method returns `true`, otherwise `false` is returned.

`void displayDetails()`

Prints to the Display Pane the `holder`, `number` and `balance` of the receiver.

`boolean equals(CurrentAccount anAccount)`

Returns `true` if receiver is equivalent to (has the same state as) the argument `anAccount`, otherwise `false` is returned.

`double getCreditLimit()`

Returns the `creditLimit` of the receiver.

`String getPinNo()`

Returns the `pinNo` of the receiver.

`void setCreditLimit(double aLimit)`

Sets the `creditLimit` of the receiver to the argument `aLimit`.

`void setPinNo(String aPin)`

Sets the `pinNo` of the receiver to the argument `aPin`.

See also the superclass `Account`.

2.4 Shapes classes

The shape classes were introduced in Unit 4.

Class Circle

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
      ∟ Circle
public class Circle extends OUAnimatedObject
```

The class `Circle` defines a shape with the characteristics of a circle.

Instance variables

```
private int diameter
private OUColour colour
private int xPos
private int yPos
```

Constructor Summary

`Circle()`

Constructor for objects of class `Circle`. Initialises `diameter` to `20`, `colour` to `OUColour.BLUE`, `xPos` to `0` and `yPos` to `0`.

Method Summary

`OUColour getColour()`

Returns the `colour` of the receiver.

`int getDiameter()`

Returns the `diameter` of the receiver.

`int getXPos()`

Returns the horizontal (`xPos`) position of the receiver.

`int getYPos()`

Returns the vertical position (`yPos`) of the receiver.

`void setColour(OUColour aColour)`

Sets the `colour` of the receiver to the value of the argument `aColour`.

`void setDiameter(int aDiameter)`

Sets the `diameter` of the receiver to the value of the argument `aDiameter`.

`void setXPos(int x)`

Sets the horizontal position (`xPos`) of the receiver to the value of the argument `x`.

`void setYPos(int y)`

Sets the vertical position (`yPos`) of the receiver to the value of the argument `y`.

`String toString()`

Returns a string representation of the receiver.

See also the superclass `OUAnimatedObject`.

Class Diamond

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
      ∟ Diamond
public class Diamond extends ou.OUAnimatedObject
```

The class `Diamond` defines a shape with the characteristics of a diamond.

Instance variables

```
private OUColour colour;
private int xPos
private int yPos
private int width
private int height
```

Constructor Summary

`Diamond()`

Constructor for objects of class `Diamond`. Initialises `colour` to `OUColour.GREEN`, `xPos` to `0`, `yPos` to `0`, `width` to `20` and `height` to `20`.

Method Summary

`OUColour getColour()`

Returns the `colour` of the receiver.

`int getHeight()`

Returns the `height` of the receiver.

`int getWidth()`

Returns the `width` of the receiver.

`int getXPos()`

Returns the horizontal position (`xPos`)of the receiver.

`int getYPos()`

Returns the vertical position (`yPos`) of the receiver.

`void setColour(OUColour aColour)`

Sets the `colour` of the receiver to the value of the argument `aColour`.

`void setHeight(int aHeight)`

Sets the `height` of the receiver to the value of the argument `aHeight`.

`void setWidth(int aWidth)`

Sets the `width` of the receiver to the value of the argument `aWidth`.

`void setXPos(int x)`

Sets the horizontal position (`xPos`) of the receiver to the value of the argument `x`.

`void setYPos(int y)`

Sets the vertical position (`yPos`) of the receiver to the value of the argument `y`.

`String toString()`

Returns a string representation of the receiver.

See also the superclass `OUAnimatedObject`.

Class Square

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
      ∟ Square
public class Square extends ou.OUAnimatedObject
```

The class `Square` defines a shape with the characteristics of a square.

Instance variables

```
private OUColour colour
private int xPos
private int yPos
private int length
```

Constructor Summary

`Square()`

Constructor for objects of class `Square`. Initialises `colour` to `OUColour.ORANGE`, `xPos` to `0`, `yPos` to `0` and `length` to `20`.

Method Summary

`OUColour getColour()`

Returns the `colour` of the receiver.

`int getLength()`

Returns the `length` of the receiver.

`int getXPos()`

Returns the horizontal position (`xPos`) of the receiver.

`int getYPos()`

Returns the vertical position (`yPos`) of the receiver.

`void setColour(OUColour aColour)`

Sets the `colour` of the receiver to the value of the argument `aColour`.

`void setLength(int aLength)`

Sets the `length` of the receiver to the value of the argument `aLength`.

`void setXPos(int x)`

Sets the horizontal position (`xPos`) of the receiver to the value of the argument `x`.

`void setYPos(int y)`

Sets the vertical position (`yPos`) of the receiver to the value of the argument `y`.

`String toString()`

Returns a string representation of the receiver.

See also the superclass `OUAnimatedObject`.

Class Triangle

```
java.lang.Object
  ∟ java.util.Observable
    ∟ ou.OUAnimatedObject
      ∟ Triangle
public class Triangle extends OUAnimatedObject
```

The class `Triangle` defines a shape with the characteristics of an isosceles triangle.

Instance variables

```
private OUColour colour
private int xPos
private int yPos
private int width
private int height
```

Constructor Summary

`Triangle()`

Constructor for objects of class `Triangle`. Initialises `colour` to `OUColour.RED`, `xPos` to `0`, `yPos` to `0`, `width` to `20` and `length` to `20`.

Method Summary

`OUColour getColour()`

Returns the `colour` of the receiver.

`int getHeight()`

Returns the `height` of the receiver.

`int getWidth()`

Returns the `width` of the receiver.

`int getXPos()`

Returns the horizontal position (`xPos`) of the receiver.

`int getYPos()`

Returns the vertical position (`yPos`) of the receiver.

`void setColour(OUColour aColour)`

Sets the `colour` of the receiver to the value of the argument `aColour`.

`void setHeight(int aHeight)`

Sets the `height` of the receiver to the value of the argument `aHeight`.

`void setWidth(int aWidth)`

Sets the `width` of the receiver to the value of the argument `aWidth`.

`void setXPos(int x)`

Sets the horizontal position (`xPos`) of the receiver to the value of the argument `x`.

`void setYPos(int y)`

Sets the vertical position (`yPos`) of the receiver to the value of the argument `y`.

`String toString()`

Returns a string representation of the receiver.

See also the superclass `OUAnimatedObject`.

Class Marionette

```
java.lang.Object
  ∟ Marionette
public class Marionette extends Object
```

Objects of class `Marionette` represent puppet-like characters made up of `Circle`, `Diamond` and `Triangle` objects.

Instance variables

```
private int xPos
private int yPos
private Diamond body
private Circle head
private Triangle rightLeg
private Triangle leftLeg
private Diamond rightArm
private Diamond leftArm
```

Constructor Summary

`Marionette()`

Constructor for objects of class `Marionette`. Initialises `xPos` to `100` and `yPos` to `100`.

Method Summary

```
void addComponents(Circle aHead, Diamond aBody, Triangle aLeftLeg,
                   Triangle aRightLeg, Diamond aLeftArm, Diamond aRightArm)
```

Sets the `head`, `body`, `leftLeg`, `rightLeg`, `leftArm` and `rightArm` of the receiver to the arguments `aHead`, `aBody`, `aLeftLeg`, `aRightLeg`, `aLeftArm`, and `aRightArm` respectively.

`void down(int increment)`

Increments the vertical position of receiver by the value of the argument `int`.

`int getXPos()`

Returns the horizontal position (`xPos`) of the receiver.

`int getYPos()`

Returns the vertical position (`yPos`) of the receiver.

`void left(int decrement)`

Decrements the horizontal position of receiver by the value of the argument `int`.

`void right(int increment)`

Increments the horizontal position of receiver by the value of the argument `int`.

`void setXPos(int newPos)`

Sets the horizontal position (`xPos`) of the receiver to the value of the argument `newPos`.

`void setYPos(int newPos)`

Sets the vertical position (`yPos`) of the receiver to the value of the argument `newPos`.

`void up(int decrement)`

Decrements the vertical position of receiver by the value of the argument.

---

3 Strings

The `String` and `StringBuilder` classes have a number of methods that are very similar and differ only in the type of their argument. In such cases, we list the method only once and give the argument as `T anArgument` which is M255 shorthand to tell you that there are multiple methods, one for each of the primitive types and one that takes an argument of type `Object`. For example:

```
static String valueOf(T anArgument)
```

Class String

```
java.lang.Object
  ∟ java.lang.String
```

All Implemented Interfaces: `Serializable`, `CharSequence`, `Comparable<String>`

```
public final class String extends Object
                  implements Serializable, Comparable<String>, CharSequence
```

The `String` class represents character strings. All string literals in Java programs, such as `"abc"`, are implemented as instances of this class. Instances of the String class are immutable, they cannot be changed once created.

Constructor summary

`String()`

Initializes a newly created empty `String` object.

`String(char[] value)`

Initializes a new `String` object so that it represents the sequence of characters currently contained in the character array argument.

`String(String original)`

Initializes a newly created `String` object so that it represents the same sequence of characters as the argument; in other words, the newly created string is a copy of the argument string.

`String(StringBuilder builder)`

Initializes a new `string` that contains the sequence of characters currently contained in the `StringBuilder` argument.

Method Summary

In the following methods, where the formal argument is of type `CharSequence`, you can assume for our purposes that the actual argument will be a `String` object.

`char charAt(int index)`

Returns the `char` value at the index specified by the argument.

`int compareTo(String anotherString)`

Compares the receiver to the argument string character by character, based on the Unicode value of each character in the strings.

If the receiver `String` comes before the argument string alphabetically the method returns a negative `int` (note that an uppercase letter comes before the same alphabetical lowercase letter).

If the receiver comes after the argument string alphabetically the method returns a positive `int`.

If the two strings are exactly equal, `0` is returned. The size of the returned `int` (positive or negative) tells you how far apart in the character sequence the first unequal characters are.

`int compareToIgnoreCase(String str)`

Compares the receiver to the argument string character by character as for the `compare()` method, but ignoring case differences.

`String concat(String str)`

Returns the receiver if the length of the argument string is `0`. Otherwise, the method returns a new `String` object that is the concatenation of the receiver followed by the argument string.

`boolean contains(CharSequence s)`

Returns `true` if the receiver contains the sequence of `char` values contained in the argument as a subsequence. `False` otherwise.

`boolean contentEquals(CharSequence cs)`

Returns `true` if the receiver contains the same sequence of `char` values as the argument. `False` otherwise.

`boolean contentEquals(StringBuffer sb)`

Returns `true` if the receiver contains the same sequence of `char` values as the `StringBuffer` argument. `False` otherwise.

`static String copyValueOf(char[] data)`

Returns a new `String` which has the same characters, in the same order as the char array argument.

`boolean endsWith(String suffix)`

Returns `true` if the receiver ends with the argument. `False` otherwise.

`boolean equals(Object anObject)`

Returns `true` if the argument is not `null` and is a `String` object that holds the same sequence of characters as the receiver. `False` otherwise.

`boolean equalsIgnoreCase(String anotherString)`

As for the `equals()` method but ignoring case considerations.

`int hashCode()`

Returns the hash code of the receiver.

In the following `indexOf()` methods, if a formal argument is given as `int ch`, you can assume for our purposes that the actual argument will be of type `char`.

`int indexOf(int ch)`

Returns the index of the first occurrence of the argument `ch` within the receiver. If the character does not occur within the receiver, `-1` is returned.

`int indexOf(int ch, int fromIndex)`

Returns the index of the first occurrence of `ch` within the receiver, starting the search at `fromIndex`. If the character does not occur within the receiver, `-1` is returned.

`int indexOf(String str)`

If the argument occurs as a substring within the receiver, then the index of the first character of the first such substring found is returned; if the argument does not occur as a substring, `-1` is returned.

`int indexOf(String str, int fromIndex)`

If the argument `str` occurs as a substring within the receiver, then the index of the first character of the first such substring found, starting the search at `fromIndex`, is returned; if the argument does not occur as a substring, `-1` is returned.

`int length()`

Returns the length of the receiver.

`String replace(char oldChar, char newChar)`

Returns a copy of the receiver where all occurrences of `oldChar` have been replaced with `newChar`.

`String[] split(String regex)`

Returns an array of strings computed by splitting the receiver around matches of the given regular expression. Any trailing empty strings will be discarded For example:

`"boo:and:foo".split(":");`

returns the array `{"boo", "and", "foo"}`

`"boo:and:foo".split("o");`

returns the array `{"b", "", ":and:f"}`

`boolean startsWith(String prefix)`

Returns `true` if the receiver starts with the argument. `False` otherwise.

`String substring(int beginIndex)`

Returns a substring of the receiver beginning at `beginIndex`.

`String substring(int beginIndex, int endIndex)`

Returns a substring of the receiver from `beginIndex` to `endIndex - 1`.

`char[] toCharArray()`

Returns an array of char which has the same characters, in the same order as the receiver.

`String toLowerCase()`

Returns a copy of the receiver with all the characters in lower case.

`String toString()`

Returns the receiver, which is already a string!

`String toUpperCase()`

Returns a copy of the receiver with all the characters in upper case.

`String trim()`

Returns a copy of the receiver, with leading and trailing whitespace omitted.

`static String valueOf(T anArgument)`

Returns the string representation of the actual argument which can be of any primitive or reference type. If `x` references an object then `String.valueOf(x)` and `x.toString()` return the same string. For this reason the `valueOf()` method is mainly used to convert values of primitive types to strings.

Class StringBuilder

```
java.lang.Object
  ∟ java.lang.StringBuilder
```

All Implemented Interfaces: `Serializable`, `CharSequence`, `Comparable<String>`

```
public final class StringBuilder extends Object
                  implements Serializable, Appendable, CharSequence
```

The `StringBuilder` class represents character strings. However unlike instances of `String`, instances of `StringBuilder` are mutable, they can be changed once created.

Constructor summary

`StringBuilder()`

Initializes a newly created empty `StringBuilder` object.

`StringBuilder(String str)`

Initializes a newly created `StringBuilder` object so that it represents the same sequence of characters as the argument;

Method Summary

`StringBuilder append(T anArgument)`

Appends a string representation of the actual argument (which can be of any primitive or reference type) to the receiver. Method returns the receiver.

`char charAt(int index)`

Returns the `char` value at the index specified by the argument.

`StringBuilder delete(int start, int end)`

Removes a substring from `start` to `end - 1`, from the receiver. Method returns the receiver.

`StringBuilder deleteCharAt(int index)`

Removes the `char` value at the index specified by the argument. Method returns the receiver.

`int indexOf(String str)`

If the argument occurs as a substring within the receiver, then the index of the first character of the first such substring found is returned; if the argument does not occur as a substring, `-1` is returned.

`int indexOf(String str, int fromIndex)`

If the argument `str` occurs as a substring within the receiver, then the index of the first character of the first such substring found, starting the search at `fromIndex`, is returned; if the argument does not occur as a substring, `-1` is returned.

`StringBuilder insert(int offset, T anArgument)`

Inserts a string representation of the second argument (which can be of any primitive or reference type) into the receiver at the index specified by `offset`. Method returns the receiver.

`int length()`

Returns the length of the receiver.

`StringBuilder replace(int start, int end, String str)`

First the characters in a substring of the receiver, which is specified as `start` to `end - 1`, are removed and then the `String` argument is inserted at `start`. Method returns the receiver.

`StringBuilder reverse()`

Reverses the order of the receiver's characters. Method returns the receiver.

`void setCharAt(int index, char ch)`

Replaces the receiver's character at `index` with `ch`.

`String substring(int start)`

Returns a `String` which is a substring of the receiver beginning at `start`.

`String substring(int start, int end)`

Returns a `String` which is a substring of the receiver from `start` to `end - 1`.

`String toString()`

Returns a `String` representation of the receiver.

4 Java Collections Framework

In this section, the type names `E`, `K` and `V` are used as placeholders for real type names that you would use in your code (such as `String`, `Frog`, `Account`, `Integer` etc.). `E` is used to indicate the declared type of a collection's elements; `K` is used to indicate the declared type of a map's keys; `V` is used to indicate the declared type of a map's values.

We have simplified the argument and return type documentation for this section in two ways. Where Sun's Javadoc for collections would show the declaration of a formal argument for a method as:

```
void putAll(Map<K, V> aMap)
```

or

```
boolean addAll(Collection<E> aCol)
```

we have simplified them to:

```
void putAll(Map aMap)
```

and

```
boolean addAll(Collection aCol)
```

Similarly for return types. We have simplified

```
SortedMap<K, V> tailMap(K fromKey)
```

to:

```
SortedMap tailMap(K fromKey)
```

4.1 Collection interfaces

Interface Collection

`java.util.Collection`

All Superinterfaces: `Iterable`

Subinterfaces include: `List`, `Queue`, `Set`, `SortedSet`

Implementing Classes include: `ArrayList`, `HashSet`, `LinkedList`, `TreeSet`

```
public interface Collection<E> extends Iterable<E>
```

> The root interface in the collection hierarchy. A collection represents a group of objects, known as its elements. Some collections allow duplicate elements and others do not. Some are ordered and others unordered. The JDK does not provide any direct implementations of this interface: it provides implementations of more specific subinterfaces like `Set` and `List`. This interface is typically used to pass collections around and manipulate them where maximum generality is desired.

Method Summary

```
boolean add(E element)
```

> Adds the argument to the receiver. Returns `true` if the operation was successful, false otherwise.

```
boolean addAll(Collection aCol)
```

> Adds all of the elements in the argument to the receiver. Returns `true` if the operation was successful, false otherwise.

```
void clear()
```

> Removes all of the elements from the receiver (if there were any).

`boolean contains(Object obj)`

Tests whether the argument is present in the receiver. Returns `true` if the argument is an element in the collection, `false` otherwise.

`boolean containsAll(Collection aCol)`

Returns `true` if the receiver contains all of the elements in the argument.

`boolean equals(Object obj)`

Compares the argument with the receiver for equality. Returns `true` if the argument object is equal to the receiver.

`int hashCode()`

Returns the hash code of the receiver.

`boolean isEmpty()`

Tests whether the receiver is empty. Returns `true` if empty, `false` otherwise.

`boolean remove(Object obj)`

Removes one occurrence of the argument from the collection. Returns `true` if the operation was successful, `false` otherwise.

`boolean removeAll(Collection aCol)`

Removes one occurrence of each of the argument's elements from the receiver. Returns `true` if the operation was successful, `false` otherwise.

`boolean retainAll(Collection aCol)`

Retains *only* the elements in the receiver that are also contained in the argument. Returns `true` if the operation was successful, `false` otherwise.

`int size()`

Returns the number of elements in the receiver.

`Object[] toArray()`

Returns an array containing all of the elements in the receiver.

`E[] toArray(E[] anArray)`

Returns an array containing all the elements in the receiver; the runtime type of the returned array is that of the argument.

Interface Iterable

`java.lang.Iterable`

Subinterfaces include: `Collection`, `List`, `Queue`, `Set`, `SortedSet`

Implementing Classes include: `ArrayList`, `HashSet`, `LinkedList`, `TreeSet`

`public interface Iterable<E>`

Implementing this interface allows a collection class's instances to be iterated over by a `foreach` statement.

Interface List

`java.util.List`

All Superinterfaces: `Collection`, `Iterable`

Implementing Classes include: `ArrayList`, `LinkedList`

`public interface List<E> extends Collection<E>`

An ordered collection (also known as a sequence). The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index (position in the list), and search for elements in the list. Unlike sets, lists typically allow duplicate elements.

Method Summary

`boolean add(E element)`

Appends the argument to the end of the receiver. Returns `true` if the operation was successful, `false` otherwise.

`void add(int index, E element)`

Inserts the argument (`element`) into the receiver at the position specified by the argument `index`. The method shifts the element currently at that position (if any) and any subsequent elements to the right (adds one to their indices).

`boolean addAll(Collection aCol)`

Appends all of the elements in the argument to the end of the receiver, in the order that they are returned by the argument's iterator. Returns `true` if the operation was successful, `false` otherwise.

`boolean addAll(int index, Collection aCol)`

Inserts all of the elements in the argument `aCol` into the receiver at the position specified by the argument `index`. Returns `true` if the operation was successful, `false` otherwise.

`boolean equals(Object obj)`

Compares the argument `obj` with the receiver for equality. Returns `true` if, and only if, the argument is also a list, both the receiver and the argument have the same size, and all corresponding pairs of elements in the two lists are equal, `false` otherwise.

`E get(int index)`

Returns the element at the position specified by the argument.

`int hashCode()`

Returns the hash code for the receiver.

`int indexOf(Object obj)`

Returns the index in the receiver of the first occurrence of the argument, or `-1` if the receiver does not contain this element.

`int lastIndexOf(Object obj)`

Returns the index in the receiver of the last occurrence of the argument, or `-1` if the receiver does not contain this element.

`E remove(int index)`

Removes and returns the element at the position specified by the argument.

`boolean remove(Object obj)`

Removes the first occurrence in the receiver of the argument. Returns `true` if the operation was successful, `false` otherwise.

`E set(int index, E element)`

Replaces the element at the position specified by the first argument with the second argument and returns the original element.

`int size()`

Returns the number of elements in the receiver.

`List subList(int fromIndex, int toIndex)`

Returns a view of a portion of the receiver between the indices specified by the arguments `fromIndex` (inclusive), and `toIndex` (exclusive). The returned list is backed by the receiver, so changes in the returned list are reflected in the receiver, and vice-versa.

`Object[] toArray()`

Returns an array containing all of the elements in the receiver in the same order.

`E[] toArray(E[] anArray)`

Returns an array containing all the elements in the receiver in the same order; the run-time type of the returned array is that of the argument.

See also the superinterface `Collection`.

Interface Set

`java.util.Set`

All Superinterfaces: `Collection`, `Iterable`

All Known Subinterfaces: `SortedSet`

Implementing Classes include: `HashSet`, `TreeSet`

`public interface Set<E> extends Collection<E>`

A collection that contains no duplicate elements, and at most one null element. As implied by its name, this interface models the mathematical *set* abstraction.

Method Summary

`boolean add(E element)`

Adds the argument to the receiver, unless it is already there. Returns `true` if the argument was added, `false` if it was not.

`boolean addAll(Collection aCol)`

Adds all of the elements in the argument to the receiver if they are not already present. Returns `true` if the operation was successful, `false` otherwise.

`boolean equals(Object obj)`

Compares the argument with the receiver for equality. Returns `true` if the argument is also a set, the two sets have the same size, and every element in the argument is also contained in the receiver, `false` otherwise.

`int hashCode()`

Returns the hash code of the receiver.

See also the superinterface `Collection`.

Interface SortedSet

`java.util.SortedSet`

All Superinterfaces: `Collection`, `Iterable`, `Set`

All Known Implementing Classes: `TreeSet`

```
public interface SortedSet<E> extends Set<E>
```

A set that keeps its elements ordered according to their natural ordering. Several additional operations are provided to take advantage of this ordering.

Method Summary

`E first()`

Returns the first (lowest) element currently in the receiver.

`SortedSet headSet(E toElement)`

Returns a view of the portion of the receiver whose elements are strictly less than `toElement`. The returned `SortedSet` is backed by the receiver, so changes in the returned `SortedSet` are reflected in the receiver, and vice-versa.

`E last()`

Returns the last (highest) element currently in the receiver.

`SortedSet subSet(E fromElement, E toElement)`

Returns a view of a portion of the receiver between the elements specified by the arguments `fromElement` (inclusive), and `toElement` (exclusive). The returned `SortedSet` is backed by the receiver, so changes in the returned `SortedSet` are reflected in the receiver, and vice-versa.

`SortedSet tailSet(E fromElement)`

Returns a view of the portion of the receiver whose elements are greater than or equal to `fromElement`. The returned `SortedSet` is backed by the receiver, so changes in the returned `SortedSet` are reflected in the receiver, and vice-versa.

See also the superinterfaces `Set` and `Collection`.

Interface Map

`java.util.Map`

Subinterfaces include: `SortedMap`

Implementing Classes include: `HashMap`, `TreeMap`

```
public interface Map<K,V>
```

A collection that maps keys to values. A map cannot contain duplicate keys; each key can map to at most one value.

Method Summary

`void clear()`

Removes all entries from the receiver.

`boolean containsKey(Object key)`

Tests whether a given key is present in the receiver. The method returns `true` if the key is present, `false` otherwise.

`boolean containsValue(Object value)`

Tests whether the argument is present as a value in the receiver. The method returns `true` if the argument is present as a value, `false` otherwise.

`boolean equals(Object obj)`

Compares the argument with the receiver for equality. Returns `true` if the argument is also a map and the two maps represent the same key-value pairs.

`V get(Object key)`

If the argument exists as a key in the receiver, then the method returns the associated value. Otherwise `null` is returned.

`int hashCode()`

Returns the hash code of the receiver.

`boolean isEmpty()`

Tests whether the receiver contains any key–value pairs. The method returns `true` if the map is empty, otherwise `false`.

`Set keySet()`

Returns a view of the keys contained in the receiver as a set. As this set is just a view of the keys in the map, if a key is removed from the returned set, then the associated key-value pair is removed from the map.

`V put(K key, V value)`

Puts a key–value pair into the receiver. If a key–value pair already exists with the same key, then the old value is overwritten by the new value. The method returns the previous value for the key, if there was one, otherwise it returns `null`.

`void putAll(Map aMap)`

Copies all of the key–value pairs from the argument into the receiver.

`V remove(Object key)`

Removes the key–value pair associated with the argument (if the key exists). Returns the previous value for the key, if there was one, otherwise null.

`int size()`

Returns the number of key-value mappings in the receiver.

`Collection values()`

Returns a view of the values contained in receiver as a collection. As this collection is just a view of the values in the map, if a value is removed from the returned collection, then the associated key-value pair is removed from the map.

Interface SortedMap

`java.util.SortedMap`

All Superinterfaces: `Map`

All Known Implementing Classes: `TreeMap`

`public interface SortedMap<K,V> extends Map<K,V>`

A map that guarantees that its elements will be in ascending key order, sorted according to the natural ordering of its keys.

Method Summary

`K firstKey()`

Returns the first (lowest) key currently in the receiver.

`SortedMap headMap(K toKey)`

Returns a view of the portion of the receiver whose keys are strictly less than `toKey`.

`K lastKey()`

Returns the last (highest) key currently in the receiver.

`SortedMap subMap(K fromKey, K toKey)`

Returns a view of the portion of the receiver whose keys range from `fromKey`, inclusive, to `toKey`, exclusive.

`SortedMap tailMap(K fromKey)`

Returns a view of the portion of the receiver whose keys are greater than or equal to `fromKey`.

See also the superinterface `Map`.

4.2 Collection implementation classes

Class HashSet

```
java.lang.Object
  ∟ java.util.AbstractCollection
      ∟ java.util.AbstractSet
        ∟ java.util.HashSet
```

All Implemented Interfaces: `Serializable`, `Cloneable`, `Iterable`, `Collection`, `Set`

```
public class HashSet<E> extends AbstractSet<E>
                        implements Set<E>, Cloneable, Serializable
```

This class implements the `Set` interface

Constructor Summary

`HashSet()`

Constructs an empty set.

`HashSet(Collection aCol)`

Constructs a set containing the elements in the argument.

Method Summary

See the superinterfaces `Set` and `Collection`

Class TreeSet

```
java.lang.Object
  ∟ java.util.AbstractCollection
    ∟ java.util.AbstractSet
      ∟ java.util.TreeSet
```

All Implemented Interfaces: `Serializable`, `Cloneable`, `Iterable`, `Collection`, `Set`, `SortedSet`

```
public class TreeSet<E> extends AbstractSet<E>
                        implements SortedSet<E>, Cloneable, Serializable
```

This class implements the `SortedSet` interface. The class guarantees that the sorted set will be in ascending element order, sorted according to the natural order of the elements.

Constructor Summary

`TreeSet()`

Constructs an empty `TreeSet`.

`TreeSet(Collection aCol)`

Constructs a `TreeSet` containing the elements in the argument, sorted according to the elements' natural order.

`TreeSet(SortedSet aSortedSet)`

Constructs a `TreeSet` containing the same elements as the `SortedSet` given by the argument.

Method Summary

See the superinterfaces `Collection` and `Set`.

Class HashMap

```
java.lang.Object
  ∟ java.util.AbstractMap
    ∟ java.util.HashMap
```

All Implemented Interfaces: `Serializable`, `Cloneable`, `Map`

```
public class HashMap<K,V> extends AbstractMap<K,V>
                          implements Map<K,V>, Cloneable, Serializable
```

Implementation of the `Map` interface.

Constructor Summary

`HashMap()`

Constructs an empty `HashMap`.

`HashMap(Map aMap)`

Constructs a `HashMap` with the same mappings as the argument.

Method Summary

See the superinterface `Map`.

Class TreeMap

```
java.lang.Object
  ∟ java.util.AbstractMap
    ∟ java.util.TreeMap
```

All Implemented Interfaces: `Serializable`, `Cloneable`, `Map`, `SortedMap`

```
public class TreeMap<K,V> extends AbstractMap<K,V>
                          implements SortedMap<K,V>, Cloneable, Serializable
```

Implementation of the `SortedMap` interface. This class guarantees that the map will sorted in ascending key order, according to the natural order of the map's keys.

Constructor Summary

`TreeMap()`

Constructs an empty map.

`TreeMap(Map aMap)`

Constructs a `TreeMap` containing the same mappings as the argument, sorted according to the keys' natural order.

`TreeMap(SortedMap aSortedMap)`

Constructs a `TreeMap` containing the same mappings as the argument, sorted according to the same ordering.

Method Summary

See the superinterfaces `Map` and `SortedMap`.

Class ArrayList

```
java.lang.Object
  ∟java.util.AbstractCollection
    ∟java.util.AbstractList
      ∟java.util.ArrayList
```

All Implemented Interfaces: `Serializable`, `Cloneable`, `Iterable`, `Collection`, `List`, `RandomAccess`

```
public class ArrayList<E> extends AbstractList<E>
                    implements List<E>, RandomAccess, Cloneable, Serializable
```

Implementation of the `List` interface.

Constructor Summary

`ArrayList()`

Constructs an empty `ArrayList`.

`ArrayList(Collection aCol)`

Constructs an `ArrayList` containing the elements of the argument, in the order they appear in the argument.

Method Summary

See the superinterfaces `List` and `Collection`.

4.3 Collection utility classes

Class Arrays

```
java.lang.Object
  ∟ java.util.Arrays
public class Arrays extends Object
```

A utility class that contains static methods for manipulating arrays (such as sorting and searching). The methods in this class throw a `NullPointerException` if the specified array reference is `null`.

Method Summary

`static List asList(E[] anArray)`

Returns a fixed-size list backed by the array argument. (Changes to the returned list 'write through' to the array.)

`static int binarySearch(int[] intArray, int anInt)`

Searches the array argument for the value specified by the second argument using the binary search algorithm. If found the method returns the index of the element, otherwise it returns (-(insertion point) - 1).

The array being searched must be sorted (as by the `sort(int)` method, below) , if not, the results are undefined. If the array contains multiple elements with the same specified value, there is no guarantee which one will be found.

There are equivalent methods for the primitive types `byte`, `char`, `double`, `float`, `long`, `short`.

`static int binarySearch(Object[] anArray, Object obj)`

Searches the array argument for the object specified by the second argument using the binary search algorithm. If found the method returns the index of the element, otherwise it returns (-(insertion point) - 1).

The array being searched must be sorted into ascending order according to the natural ordering of its elements (as by the `sort(Object[]` method, below), if not, the results are undefined. If the array contains multiple elements equal to the specified object, there is no guarantee which one will be found.

`static boolean deepEquals(Object[] anArray1, Object[] anArray2)`

Returns `true` if the two array arguments are deeply equal to one another. Two array references are considered deeply equal if both are `null`, or if they refer to arrays that contain the same number of elements and all corresponding pairs of elements in the two arrays are deeply equal.

`static int deepHashCode(Object[] anArray)`

Returns a hash code based on the 'deep contents' of the array argument. If the array contains other arrays as elements, the hash code is based on their contents and so on, *ad infinitum*.

`static String deepToString(Object[] anArray)`

Returns a string representation of the 'deep contents' of the array argument. If the array contains other arrays as elements, the string representation contains their contents and so on.

`static boolean equals(int[] anArray, int[] anArray2)`

Returns `true` if the two array arguments are equal to one another, `false` otherwise. There are equivalent methods for the primitive types `boolean`, `byte`, `char`, `double`, `float`, `long`, `short`.

`static boolean equals(Object[] anArray, Object[] anArray2)`

Returns `true` if the two array arguments are equal to one another, `false` otherwise.

`static void fill(int[] anArray, int val)`

Assigns the `int` argument `val` to each component of the argument array. There are equivalent methods for the primitive types `boolean`, `byte`, `char`, `double`, `float`, `long`, `short`.

`static void fill(int[] anArray, int fromIndex, int toIndex, int val)`

Assigns the `int` argument `val` to the components of a sub-array of the array argument. The range of the sub-array is given by the arguments `fromIndex` (inclusive) and `toIndex` (exclusive). There are equivalent methods for the primitive types `boolean`, `byte`, `char`, `double`, `float`, `long`, `short`.

`static void fill(Object[] anArray, int fromIndex, int toIndex, Object val)`

Assigns the `Object` argument `val` to the components of a sub-array of the array argument. The range of the sub-array is given by the arguments `fromIndex` (inclusive) and `toIndex` (exclusive).

`static void fill(Object[] anArray, Object val)`

Assigns the `Object` argument `val` to each component of the array argument.

`static int hashCode(int[] anArray)`

Returns a hash code based on the contents of the array argument. There are equivalent methods for the primitive types `boolean`, `byte`, `char`, `double`, `float`, `long`, `short`.

`static int hashCode(Object[] anArray)`

Returns a hash code based on the contents of the array argument.

`static void sort(int[] anArray)`

Sorts the argument, an array of integers into ascending numerical order. There are equivalent methods for the primitive types: `byte`, `char`, `double`, `float`, `long`, `short`.

`static void sort(int[] anArray, int fromIndex, int toIndex)`

Sorts a sub-array of the array argument, into ascending numerical order. The sub-array is specified by the arguments `fromIndex` (inclusive) and `toIndex` (exclusive). There are equivalent methods for the primitive types: `byte`, `char`, `double`, `float`, `long`, `short`.

`static void sort(Object[] anArray)`

Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.

`static void sort(Object[] anArray, int fromIndex, int toIndex)`

Sorts a sub-array of the array argument into ascending order, according to the natural ordering of its elements. The sub-array is specified by the arguments `fromIndex` (inclusive) and `toIndex` (exclusive).

`static String toString(int[] anArray)`

Returns a string representation of the contents of the argument array. There are equivalent methods for the primitive types `boolean`, `byte`, `char`, `double`, `float`, `long`, `short`.

`static String toString(Object[] anArray)`

Returns a string representation of the contents of the argument array.

Class Collections

```
java.lang.Object
  ∟ java.util.Collections
public class Collections extends Object
```

A utility class consisting exclusively of static methods that operate on, or return, collections. The methods of this class all throw a `NullPointerException` if the objects provided to them as arguments are `null`.

Method Summary

`static boolean disjoint(Collection col1, Collection col2)`

Returns `true` if the two arguments have no elements in common, `false` otherwise.

`static int frequency(Collection aCol, Object obj)`

Returns the number of elements in the argument `aCol` that are equal to the argument `obj`.

`static int indexOfSubList(List source, List subList)`

If `sublist` is a sub-list of `source` the method returns the index of the first element of `subList` within `source`, otherwise `-1` is returned.

`static int lastIndexOfSubList(List source, List subList)`

If `sublist` is a sub-list of `source` the method returns the index of the last element of `subList` within `source`, otherwise `-1` is returned.

`static void rotate(List list, int distance)`

Rotates the elements in the specified list by the specified `distance`.

`static void shuffle(List list)`

Randomly shuffles the arguments elements.

`public static E max(Collection coll)`

Returns the maximum element in the argument, according to the natural ordering of its elements. All elements in the collection must implement the `Comparable` interface.

`public static E min(Collection coll)`

Returns the minimum element in the argument, according to the natural ordering of its elements. All elements in the collection must implement the `Comparable` interface.

`public static void reverse(List list)`

Reverses the order of the elements in the argument.

`public static void sort(List list)`

Sorts the argument into ascending order, according to the natural ordering of its elements. All elements in the list must implement the `Comparable` interface.

`public static void swap(List list, int i, int j)`

Swaps the elements of the argument that are at indices `i` and `j`.

---

5 Files and streams

This section contains the classes for writing to and reading from files.

5.1 Files and pathnames

Class File

```
java.lang.Object
  ∟ java.io.File
public class File extends Object
                implements Serializable, Comparable<File>
```

An abstract representation of file and directory pathnames.

Constructor Summary

`File(String pathname)`

Constructs a `File` instance by converting the given pathname string into an abstract pathname.

Method Summary

`boolean canRead()`

Returns `true` if the application can read the file denoted by the receiver, otherwise `false`.

`boolean canWrite()`

Returns `true` if the application can modify the file denoted by the receiver, otherwise `false`.

`boolean exists()`

Returns `true` if the file or directory denoted by the receiver exists, otherwise `false`.

`boolean isDirectory()`

Returns `true` if the file denoted by the receiver is a directory (folder), otherwise `false`.

`boolean isFile()`

Returns `true` if the file denoted by the receiver is a normal file, otherwise `false`.

`String toString()`

Returns the pathname string of the receiver.

5.2 Reading from character streams

Class Reader

```
java.lang.Object
  ∟ java.io.Reader
public abstract class Reader extends Object implements Readable, Closeable
```

Abstract class for reading character streams. Instances of subclasses of the `Reader` class handle (16 bit) character streams; this means that they correctly handle textual information based on characters and strings.

Method Summary

```
abstract void close() throws IOException
```

Closes the stream.

```
int read() throws IOException
```

Reads a single character. Returns the character read, as an integer in the range 0 to 65535, or `-1` if the end of the stream has been reached.

```
int read(char[] cbuf) throws IOException
```

Reads characters into an array specified by the argument `cbuf`. Returns the number of characters read, or `-1` if the end of the stream has been reached.

```
abstract int read(char[] cbuf, int offSet, int length) throws IOException
```

Reads characters into a portion of `cbuf` storing the first character at `offSet` and reading `length` characters. Returns the number of characters read, or `-1` if the end of the stream has been reached.

```
int read(CharBuffer target) throws IOException
```

Attempts to read characters into the specified character buffer `target`. Returns the number of characters added to the buffer, or `-1` if this source of characters is at its end.

```
long skip(long n) throws IOException
```

Skips `n` characters.

Class FileReader

```
java.lang.Object
  ∟ java.io.Reader
    ∟ java.io.InputStreamReader
      ∟ java.io.FileReader
public class FileReader extends InputStreamReader
```

The simplest `Reader` subclass to use to open an input stream to read characters from a text file.

Constructor Summary

```
FileReader(File file) throws FileNotFoundException
```

Constructs a new `FileReader`, given the `File` to read from.

Method Summary

See the `Reader` abstract class.

Class BufferedReader

```
java.lang.Object
  ∟ java.io.Reader
    ∟ java.io.BufferedReader
public class BufferedReader extends Reader
```

Reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays and lines. In general, each read request made of a `Reader` causes a corresponding read request to be made of the underlying character or byte stream. It is therefore advisable to use a `BufferedReader` to wrap any instance of a subclass of `Reader` whose `read()` operations may be costly, such as instances of `FileReader`.

Constructor Summary

`BufferedReader(Reader in)`

Constructs a buffered character-input stream.

Method Summary

`String readLine() throws IOException`

Reads a line of text. A line is considered to be terminated by any one of a linefeed (`'\n'`), a carriage return (`'\r'`), or a carriage return followed immediately by a linefeed. Returns a `String` containing the contents of the line, not including any line-termination characters, or `null` if the end of the stream has been reached.

See also the `Reader` abstract class.

Class Scanner

```
java.lang.Object
  ∟ java.util.Scanner
public final class Scanner extends Object implements Iterator<String>
```

A simple text scanner which can parse primitive types and strings using regular expressions. A `Scanner` breaks its input into tokens using a delimiter pattern, which by default matches whitespace. The resulting tokens may then be converted into values of different types using the various `next` methods.

Constructor Summary

`Scanner(File source) throws FileNotFoundException`

Constructs a `Scanner` that produces values scanned from the specified file.

`Scanner(Readable source)`

Constructs a `Scanner` that produces values scanned from the specified source. Note that the `Reader` class implements the `Readable` interface so subclasses of `Reader` can be wrapped by a scanner.

`Scanner(String source)`

Constructs a `Scanner` that produces values scanned from the specified string.

Method Summary

`void close()`

Closes the receiver.

`Pattern delimiter()`

Returns the `Pattern` the receiver is currently using to match delimiters.

`boolean hasNext()`

Returns `true` if the receiver has another token in its input.

`boolean hasNextInt()`

Returns `true` if the next token in the receiver's input can be interpreted as an `int` value, `false` otherwise.

The following similar methods are also available: `hasNextBigDecimal()`, `hasNextBigInteger()`, `hasNextBoolean()`, `hasNextByte()`, `hasNextDouble()`, `hasNextFloat()`, `hasNextLong()`, `hasNextShort()`

`boolean hasNextLine()`

Returns `true` if there is another line in the input of the receiver.

`String next()`

Finds and returns the next complete token from the receiver.

`int nextInt()`

Scans and returns the next token of the input as an `int`.

In addition the following similar methods are also available: `nextBigDecimal()`, `nextBigInteger()`, `nextBoolean()`, `nextByte()`, `nextDouble()`, `nextFloat()`, `nextLong()`, `nextShort()`

`String nextLine()`

Advances the receiver past the current line and returns the input that was skipped as a string.

`Scanner useDelimiter(String pattern)`

Sets the receiver's delimiting pattern to a pattern constructed from the specified `String`. Returns this scanner.

5.3 Writing to character streams

Class Writer

```
java.lang.Object
  ∟ java.io.Writer
public abstract class Writer extends Object
                              implements Appendable, Closeable, Flushable
```

Abstract class for writing to character streams. Instances of subclasses of the `Writer` class handle (16 bit) character streams; this means that they correctly handle textual information based on characters and strings.

Method Summary

In the following methods, where the formal argument is of type `CharSequence` you can assume for our purposes that the actual argument will be a `String` object or a `StringBuider` object.

```
Writer append(char c) throws IOException
```

Appends the argument character `c` to the receiver. Returns the receiver.

```
Writer append(CharSequence csq) throws IOException
```

Appends the argument character sequence `csq` to the receiver. Returns the receiver.

```
Writer append(CharSequence csq, int start, int end) throws IOException
```

Appends a subsequence of the argument character sequence `csq` to the receiver, where `start` is the index of the first character in the subsequence and `end` is the index of the character following the last character. Returns the receiver.

```
abstract void close() throws IOException
```

Closes the stream, flushing it first.

```
abstract void flush() throws IOException
```

Flushes the stream. If the stream has saved any characters from the various `write()` methods in a buffer, they are written immediately to their intended destination. Then, if that destination is another character or byte stream it is flushed. Thus one `flush()` invocation will flush all the buffers in a chain of `Writer`s and `OutputStream`s. If the intended destination of this stream is an abstraction provided by the underlying operating system, for example a file, then flushing the stream guarantees only that bytes previously written to the stream are passed to the operating system for writing; it does not guarantee that they are actually written to a physical device such as a disk drive.

```
void write(char[] cbuf) throws IOException
```

Writes the argument array of characters `cbuf`.

```
abstract void write(char[] cbuf, int offSet, int length) throws
                                                          IOException
```

Writes `length` characters of the array `cbuf` starting with the character in index position `offSet`.

```
void write(int c) throws IOException
```

Writes a single character.

```
void write(String str) throws IOException
```

Writes a string.

`void write(String str, int off, int len) throws IOException`

Writes `length` characters of the string `str` starting with the character in index position `offSet`.

Class FileWriter

```
java.lang.Object
  ∟ java.io.Writer
    ∟ java.io.OutputStreamWriter
      ∟ java.io.FileWriter
public class FileWriter extends OutputStreamWriter
```

The simplest `Writer` subclass to use to open an output stream to write characters to a text file.

Constructor Summary

`FileWriter(File file) throws IOException`

Constructs a `FileWriter` object given a `File` object. Anything written to `file` will be added at the beginning, so overwriting any existing contents.

`FileWriter(File file boolean append) throws IOException`

Constructs a `FileWriter` object given a `File` object. If `append` is `true` then anything written to `file` will be added at the end. If `append` is `false`, then anything written to `file` will be added at the beginning, so overwriting any existing contents.

Method Summary

See the `Writer` abstract class.

Class BufferedWriter

```
java.lang.Object
  ∟ java.io.Writer
    ∟ java.io.BufferedWriter
public class BufferedWriter extends Writer
```

Writes text to a character-output stream, buffering characters so as to provide for the efficient writing of single characters, arrays and strings. It is advisable to wrap a `BufferedWriter` around any `Writer` whose `write()` operations may be costly, such as instances of `FileWriter`.

Constructor Summary

`BufferedWriter(Writer out)`

Constructs a buffered character-output stream that uses a default-sized output buffer.

Method Summary

`void newLine() throws IOException`

Writes a line separator. This method uses the platform's own notion of line separator as defined by the system property `line.separator`. Using this method to terminate each output line is therefore preferred to writing a newline character directly.

See also the `Writer` abstract class.

5.4 Reading from byte streams

Class InputStream

```
java.lang.Object
  ∟ java.io.InputStream
public abstract class InputStream extends Object implements Closeable
```

This abstract class is the superclass of all classes whose instances represent an input stream of bytes such as `FileInputStream` and `ObjectInputStream`.

Method Summary

`void close() throws IOException`

Closes this input stream and releases any system resources associated with it.

`abstract int read() throws IOException`

Reads the next `byte` of data from the input stream.

`int read(byte[] byteArray) throws IOException`

Reads up to `byteArray.length` number of bytes from the input stream and stores them into `byteArray`.

`int read(byte[] byteArray, int offSet, int length) throws IOException`

Starting from `offSet`, reads up to `length` bytes of data from the input stream into `byteArray`.

`long skip(long n) throws IOException`

Skips over and discards `n` bytes of data from this input stream.

Class FileInputStream

```
java.lang.Object
  ∟ java.io.InputStream
    ∟ java.io.FileInputStream
public class FileInputStream extends InputStream
```

Instances of `FileInputStream` are used for reading streams of raw bytes from a file. For reading streams of characters, consider using `FileReader`.

Constructor Summary

`FileInputStream(File file) throws FileNotFoundException`

Constructs a `FileInputStream` by opening a connection to an actual file – the file named by the `File` object file in the file system.

Method Summary

See the abstract class `InputStream`.

Class BufferedInputStream

```
java.lang.Object
  ∟ java.io.InputStream
    ∟ java.io.FilterInputStream
      ∟ java.io.BufferedInputStream
public class BufferedInputStream extends FilterInputStream
```

A `BufferedInputStream` adds the ability to buffer the input of another input stream. When the `BufferedInputStream` is created, an internal buffer array is created. As bytes from the stream are read or skipped, the internal buffer is refilled as necessary from the contained input stream, many bytes at a time.

Constructor Summary

`BufferedInputStream(InputStream in)`

Constructs a `BufferedInputStream` that wraps the input stream `in`.

Method Summary

See also the abstract class `InputStream`.

Class ObjectInputStream

```
java.lang.Object
  ∟ java.io.InputStream
    ∟ java.io.ObjectInputStream
public class ObjectInputStream extends InputStream
                      implements ObjectInput, ObjectStreamConstants
```

An `ObjectInputStream` deserialises primitive data and objects previously written using an `ObjectOutputStream`.

Constructor Summary

`ObjectInputStream(InputStream in) throws IOException`

Constructs an `ObjectInputStream` that reads from the specified `InputStream`.

Method Summary

`int readInt() throws IOException`

Reads an `int` from the `ObjectInputStream`.

In addition the following similar methods are also available: `readBoolean()`, `readByte()`, `readChar()`, `readDouble()`, `readFloat()`, `readLong(long)`, `readShort()`

`Object readObject() throws IOException, ClassNotFoundException`

Reads an object from the `ObjectInputStream`.

See also the abstract class `InputStream`.

5.5 Writing to byte streams

Class OutputStream

```
java.lang.Object
  ∟ java.io.OutputStream
public abstract class OutputStream extends Object
                                    implements Closeable, Flushable
```

This abstract class is the superclass of all classes whose instances represent an output stream of bytes, such as `FileOutputStream` and `ObjectOutputStream`.

Method Summary

`void close() throws IOException`

Closes this output stream and releases any system resources associated with it.

`void flush() throws IOException`

Flushes this output stream and forces any buffered output bytes to be written out.

`void write(byte[] byteArray) throws IOException`

Writes `byteArray.length` bytes from `byteArray` to this output stream.

`void write(byte[] byteArray, int offSet, int length) throws IOException`

Starting from `offSet`, writes `length` bytes of data from `byteArray` to this output stream.

`abstract void write(int aByte) throws IOException`

Writes the argument to this output stream.

Class FileOutputStream

```
java.lang.Object
  ∟ java.io.OutputStream
    ∟ java.io.FileOutputStream
public class FileOutputStream extends OutputStream
```

Instances of `FileOutputStream` are used for writing streams of raw bytes to a file. For writing streams of characters, consider using `FileWriter`.

Constructor Summary

`FileOutputStream(File file) throws FileNotFoundException`

Constructs a file output stream to write to the file represented by the argument.

Method Summary

See the abstract class `OutputStream`.

Class BufferedOutputStream

```
java.lang.Object
  ∟ java.io.OutputStream
    ∟ java.io.FilterOutputStream
      ∟ java.io.BufferedOutputStream
public class BufferedOutputStream extends FilterOutputStream
```

The class implements a buffered output stream. By setting up such an output stream, an application can write bytes to the underlying output stream without necessarily causing a call to the underlying system for each byte written.

Constructor Summary

`BufferedOutputStream(OutputStream out)`

Constructs a `BufferedOutputStream` that wraps the specified underlying output stream `out`.

Method Summary

See the abstract class `OutputStream`.

Class ObjectOutputStream

```
java.lang.Object
  ∟ java.io.OutputStream
    ∟ java.io.ObjectOutputStream
public class ObjectOutputStream extends OutputStream
                    implements ObjectOutput, ObjectStreamConstants
```

Instances of `ObjectOutputStream` are used to write primitive values and objects to an `OutputStream`. The objects can be read (reconstituted) using an `ObjectInputStream`.

Constructor Summary

`ObjectOutputStream(OutputStream out) throws IOException`

Constructs an `ObjectOutputStream` that writes to the specified `OutputStream`.

Method Summary

`void writeInt(int val) throws IOException`

Writes the argument to the `ObjectOutputStream`.

In addition the following similar methods are also available:
`writeBoolean(boolean)`, `writeByte(int)`, `writeChar(int)`, `writeDouble(double)`, `writeFloat(float)`, `writeLong(long)`, `writeShort(short)`

`void writeObject(Object obj) throws IOException`

Writes the argument to the `ObjectOutputStream`.

See the abstract class `OutputStream`.

5.6 Serialisation

Interface Serializable

`java.io.Serializable`

`public interface Serializable`

Serialisation of instances of a class is enabled by the class implementing the `java.io.Serializable` interface. Classes that do not implement this interface will not have any of their state serialised or deserialised. All subclasses of a class that implements `Serializable` are themselves serialisable. The `Serializable` interface has no methods or instance variables and serves only to identify the semantics of being serialisable.

---

6 Exceptions

Class Throwable

```
java.lang.Object
  ∟ java.lang.Throwable
public class Throwable extends Object implements Serializable
```

The `Throwable` class is the superclass of all errors and exceptions in the Java language. Only objects that are instances of this class (or one of its subclasses) are thrown by the Java Virtual Machine or can be thrown by the Java `throw` statement. Similarly, only this class or one of its subclasses can be the argument type in a `catch` clause.

Method Summary

`Throwable getCause()`

Returns the cause of this throwable or `null` if the cause is nonexistent or unknown.

`String getMessage()`

Returns the detailed message string of this throwable.

`String toString()`

Returns a short description of this throwable.

Class Exception

```
java.lang.Object
  ∟ java.lang.Throwable
      ∟ java.lang.Exception
public class Exception extends Throwable
```

The class `Exception` and its subclasses are a form of `Throwable` that indicates conditions that a reasonable application might want to catch. All direct subclasses of `Exception` (except `RuntimeException` and its subclasses) are checked exceptions.

Method Summary

See the `Throwable` class.

6.1 Checked exceptions

Class ClassNotFoundException

```
java.lang.Object
  ∟ java.lang.Throwable
    ∟ java.lang.Exception
      ∟ java.lang.ClassNotFoundException
public class ClassNotFoundException extends Exception
```

Thrown when an application tries to load in a class through its string name using the `forName` method in class `Class`, the `findSystemClass` method in class `ClassLoader` or the `loadClass` method in class `ClassLoader`, but no definition for the class with the specified name could be found.

Method Summary

See the `Throwable` class.

Class FileNotFoundException

```
java.lang.Object
  ∟ java.lang.Throwable
    ∟ java.lang.Exception
      ∟ java.io.IOException
        ∟ java.io.FileNotFoundException
public class FileNotFoundException extends IOException
```

Signals that an attempt to open the file denoted by a specified pathname has failed. This exception will be thrown by the `FileInputStream` and `FileOutputStream` constructors when a file with the specified pathname does not exist. It will also be thrown by these constructors if the file does exist but for some reason is inaccessible, for example when an attempt is made to open a read-only file for writing.

Method Summary

See the `Throwable` class.

Class IOException

```
java.lang.Object
  ∟ java.lang.Throwable
    ∟ java.lang.Exception
      ∟ java.io.IOException
public class IOException extends Exception
```

Signals that an I/O exception of some sort has occurred. This class is the general class of exceptions produced by failed or interrupted I/O operations.

Method Summary

See the `Throwable` class.

6.2 Unchecked exceptions

Class RuntimeException

```
java.lang.Object
  ∟java.lang.Throwable
    ∟java.lang.Exception
      ∟java.lang.RuntimeException
public class RuntimeException extends Exception
```

`RuntimeException` is the superclass of those exceptions that can be thrown during the normal operation of the Java Virtual Machine. A method is not required to catch any instances of subclasses of `RuntimeException` that might be thrown during the execution of that method as these exceptions are unchecked exceptions.

Method Summary

See the `Throwable` class.

Class ArithmeticException

```
java.lang.Object
  ∟java.lang.Throwable
    ∟java.lang.Exception
      ∟java.lang.RuntimeException
        ∟java.lang.ArithmeticException
public class ArithmeticException extends RuntimeException
```

Thrown when an exceptional arithmetic condition has occurred. For example, an integer 'divide by zero' throws an instance of this class.

Method Summary

See the `Throwable` class.

Class NullPointerException

```
java.lang.Object
  ∟ java.lang.Throwable
    ∟ java.lang.Exception
      ∟ java.lang.RuntimeException
        ∟ java.lang.NullPointerException
public class NullPointerException extends RuntimeException
```

Thrown when an application attempts to use `null` in a case where an object is required.

Method Summary

See the `Throwable` class.

Class IllegalArgumentException

```
java.lang.Object
  ∟ java.lang.Throwable
    ∟ java.lang.Exception
      ∟ java.lang.RuntimeException
        ∟ java.lang.IllegalArgumentException
public class IllegalArgumentException extends RuntimeException
```

Thrown to indicate that a method has been passed an illegal or inappropriate argument.

Method Summary

See the `Throwable` class.

Class NumberFormatException

```
java.lang.Object
  ∟ java.lang.Throwable
    ∟ java.lang.Exception
      ∟ java.lang.RuntimeException
        ∟ java.lang.IllegalArgumentException
          ∟ java.lang.NumberFormatException
public class NumberFormatException extends IllegalArgumentException
```

Thrown to indicate that the executing method has attempted to convert a string to one of the numeric types, but that the string does not have the appropriate format.

Method Summary

See the `Throwable` class.

Class ArrayIndexOutOfBoundsException

```
java.lang.Object
  ∟ java.lang.Throwable
    ∟ java.lang.Exception
      ∟ java.lang.RuntimeException
        ∟ java.lang.IndexOutOfBoundsException
          ∟ java.lang.ArrayIndexOutOfBoundsException
public class ArrayIndexOutOfBoundsException extends IndexOutOfBoundsException
```

Thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.

Method Summary

See the `Throwable` class.

Class StringIndexOutOfBoundsException

```
java.lang.Object
  ∟ java.lang.Throwable
    ∟ java.lang.Exception
      ∟ java.lang.RuntimeException
        ∟ java.lang.IndexOutOfBoundsException
          ∟ java.lang.StringIndexOutOfBoundsException
public class StringIndexOutOfBoundsException extends IndexOutOfBoundsException
```

Thrown to indicate that a string has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the string.

Method Summary

See the `Throwable` class.

---

7 Java operators used in M255

The Java operators used in M255, shown in order of precedence – from highest to lowest.

Prioriy	Operators	Precedence
1	postfix	`++ --`
2	unary	`- !`
3	creation	`new`
4	multiplicative	`* / %`
5	additive	`+ -`
6	relational	`< > <= >= instanceof`
7	equality	`== !=`
8	logical AND	`&&`
9	logical OR	`||`
10	assignment	`=`

Index

Classes shown in **bold**, methods shown in `code style`

A

Account 11
`Account()` 12
`Account(String, String, double)` 12
`add(E)` 25, 27, 28
`add(int, E)` 27
`addAll(Collection)` 25, 27, 28
`addAll(int, Collection)` 27
`addComponents(Circle, Diamond, Triangle, Triangle, Diamond, Diamond)` 18
`alert(String)` 5
Amphibian 7
`append(char)` 41
`append(CharSequence)` 41
`append(CharSequence, int, int)` 41
`append(T)` 23
ArithmeticException 50
ArrayIndexOutOfBoundsException 51
ArrayList 33
`ArrayList()` 33
`ArrayList(Collection)` 33
Arrays 33
`asList(E[])` 34
`availableToSpend()` 13

B

BarnDanceCaller 10
`BarnDanceCaller()` 10
`binarySearch(int[], int)` 34
`binarySearch(Object[], Object)` 34
`brown()` 7
BufferedInputStream 44
`BufferedInputStream(InputStream)` 44
BufferedOutputStream 46
`BufferedOutputStream(OutputStream)` 46
BufferedReader 39
`BufferedReader(Reader)` 39
BufferedWriter 42
`BufferedWriter(Writer)` 42

C

`canRead()` 37
`canWrite()` 37
`charAt(int)` 20, 23
`checkPin(String)` 13
Circle 14
`Circle()` 14
ClassNotFoundException 48
`clear()` 25, 29
`close()` 38, 40, 41, 43, 45
Collection 25
Collections 36
`compareTo(String)` 21
`compareToIgnoreCase(String)` 21
`concat(String)` 21
`confirm(String)` 5
`contains(CharSequence)` 21
`contains(Object)` 26
`containsAll(Collection)` 26
`containsKey(Object)` 29
`containsValue(Object)` 29
`contentEquals(CharSequence)` 21
`contentEquals(StringBuffer)` 21
`copyValueOf(char[])` 21
`credit(double)` 12
`croak()` 7
CurrentAccount 13
`CurrentAccount()` 13
`CurrentAccount(String, String, double, double, String)` 13

D

`dance1(int)` 10
`dance2(int)` 11
`dance3(int)` 11
`debit(double)` 12, 13
`deepEquals(Object[], Object[])` 34
`deepHashCode(Object[])` 34
`deepToString(Object[])` 34
`delete(int, int)` 23
`deleteCharAt(int)` 23
`delimiter()` 40
Diamond 15
`Diamond()` 15

disjoint(Collection, Collection) 36
displayDetails() 13
down() 9
down(int) 18
downBy(int) 9

E

endsWith(String) 21
equals(Account) 12
equals(CurrentAccount) 13
equals(int[], int[]) 34
equals(Object) 21, 26, 27, 28, 30
equals(Object[], Object[]) 34
equalsIgnoreCase(String) 21
Exception 48
exists() 37

F

File 37
File(String) 37
FileInputStream 43
FileInputStream(File) 43
FileNotFoundException 49
FileOutputStream 45
FileOutputStream(File) 45
FileReader 38
FileReader(File) 38
FileWriter 42
FileWriter(File) 42
fill(int[], int) 34
fill(int[], int, int, int) 34
fill(Object[], int, int, Object) 35
fill(Object[], Object) 35
first() 29
firstKey() 30
flush() 41, 45
frequency(Collection, Object) 36
Frog 8
Frog() 8

G

get(int) 27
get(Object) 30
getBalance() 12
getCause() 48
getColour() 7, 14, 15, 16, 17
getCreditLimit() 13
getDancer1() 11
getDancer2() 11
getDiameter() 14
getFilename() 6
getFilename(String) 6
getHeight() 9, 15, 17
getHolder() 12
getLength() 16
getMessage() 48
getNumber() 12
getPinNo() 13
getPosition() 7
getWidth() 15, 17
getXPos() 14, 15, 16, 17, 18
getYPos() 14, 15, 16, 17, 18
green() 7

H

hashCode() 21, 26, 27, 28, 30
hashCode(int[]) 35
hashCode(Object[]) 35
HashMap 32
HashMap() 32
HashMap(Map) 32
HashSet 31
HashSet() 31
HashSet(Collection) 31
hasNext() 40
hasNextBigDecimal() 40
hasNextBigInteger() 40
hasNextBoolean() 40
hasNextByte() 40
hasNextDouble() 40
hasNextFloat() 40
hasNextInt() 40
hasNextLine() 40
hasNextLong() 40
hasNextShort() 40
headMap(K) 31
headSet(E) 29
home() 7, 8, 9, 10
HoverFrog 9
HoverFrog() 9

I
IllegalArgumentException 50
indexOf(int) 21
indexOf(int, int) 22
indexOf(Object) 27
indexOf(String) 22, 24
indexOf(String, int) 22, 24
indexOfSubList(List, List) 36
InputStream 43
insert(int, T) 24
instanceof 52
IOException 49
isDirectory() 37
isEmpty() 26, 30
isFile() 37
Iterable 26

J
jump() 8

K
keySet() 30

L
last() 29
lastIndexOf(Object) 27
lastIndexOfSubList(List, List) 36
lastKey() 31
left() 7, 8, 10
left(int) 18
length() 22, 24
List 27

M
Map 29
Marionette 18
Marionette() 18
max(Collection) 36
min(Collection) 36

N
new 52
newLine() 42
next() 40
nextBigDecimal() 40
nextBigInteger() 40
nextBoolean() 40
nextByte() 40
nextDouble() 40
nextFloat() 40
nextInt() 40
nextLine() 40
nextLong() 40
nextShort() 40
NullPointerException 50
NumberFormatException 51

O
ObjectInputStream 44
ObjectInputStream(InputStream) 44
ObjectOutputStream 46
ObjectOutputStream(OutputStream) 46
OUAnimatedObject 4
OUColour 4
OUColour(int, int, int) 5
OUDialog 5
OUFileChooser 6
OutputStream 45

P
performAction(String) 4
put(K,V) 30
putAll(Map) 30

R
read() 38, 43
read(byte[]) 43
read(byte[], int, int) 43
read(char[]) 38
read(char[], int, int) 38
read(CharBuffer) 38
readBoolean() 44
readByte() 44
readChar() 44
readDouble() 44
Reader 38
readFloat() 44
readInt() 44
readLine() 39
readLong(long) 44

readObject() 44
readShort() 44
remove(int) 27
remove(Object) 26, 28, 30
removeAll(Collection) 26
replace(char, char) 22
replace(int, int, String) 24
request(String) 5
request(String, String) 5
retainAll(Collection) 26
reverse() 24
reverse(List) 36
right() 7, 8, 10
right(int) 18
rotate(List, int) 36
RuntimeException 49

S

sameColourAs(Amphibian) 7
Scanner 39
Scanner(File) 39
Scanner(Readable) 39
Scanner(String) 39
Serializable 47
Set 28
set(int, E) 28
setBalance(double) 12
setCharAt(int, char) 24
setColour(OUColour) 8, 14, 15, 16, 17
setCreditLimit(double) 13
setDancer1(Frog) 11
setDancer2(Frog) 11
setDancers(Frog, Frog) 11
setDiameter(int) 14
setHeight(int) 9, 15, 17
setHolder(String) 12
setLength(int) 16
setNumber(String) 12
setPinNo(String) 14
setPosition(int) 8
setWidth(int) 15, 17
setXPos(int) 14, 15, 16, 17, 19
setYPos(int) 14, 16, 18, 19
shuffle(List) 36
size() 26, 28, 30
skip(long) 38, 43
sort(int[]) 35
sort(int[], int, int) 35
sort(List) 36
sort(Object[]) 35
sort(Object[], int, int) 35
SortedMap 30
SortedSet 29
split(String) 22
Square 16
Square() 16
startsWith(String) 22
String 20
String() 20
String(char[]) 20
String(String) 20
String(StringBuilder) 20
StringBuilder 23
StringBuilder() 23
StringBuilder(String) 23
StringIndexOutOfBoundsException 51
subList(int, int) 28
subMap(K, K) 31
subSet(E, E) 29
substring(int) 22, 24
substring(int, int) 22, 24
swap(List, int, int) 36

T

tailMap(K) 31
tailSet(E) 29
takeUpYourPositions() 11
Throwable 48
Toad 10
Toad() 10
toArray() 26, 28
toArray(E[]) 26, 28
toCharArray() 22
toLowerCase() 22
toString() 5, 8, 9, 15, 16, 17, 18, 23, 24, 37, 48
toString(int[]) 35
toString(Object[]) 35
toUpperCase() 23
transfer(Account, double) 12

TreeMap 32
TreeMap() 33
TreeMap(Map) 33
TreeMap(SortedMap) 33
TreeSet 31
TreeSet() 32
TreeSet(Collection) 32
TreeSet(SortedSet) 32
Triangle 17
Triangle() 17
trim() 23

U

up() 9
up(int) 19
upBy(int) 9
update(String) 4
useDelimiter(String) 40

V

valueOf(T) 23
values() 30

W

write(byte[]) 45
write(byte[], int, int) 45
write(char[]) 41
write(char[], int, int) 41
write(int) 41, 45
write(String) 41
write(String, int, int) 42
writeBoolean(boolean) 46
writeByte(int) 46
writeChar(int) 46
writeDouble(double) 46
writeFloat(float) 46
writeInt(int) 46
writeLong(long) 46
writeObject(Object) 46
Writer 41
writeShort(short) 46

Printed in the United Kingdom by
The Open University